

MODULE 2

Syntax Analysis

Role of the Syntax Analyzer – Syntax error handling, Review of Context Free Grammars – Derivation and Parse Trees, Eliminating Ambiguity, Basic parsing approaches – Eliminating left recursion, left factoring. Top-Down Parsing – Recursive Descent parsing, Predictive Parsing, LL(1) Grammars.

2. SYNTAX ANALYSIS

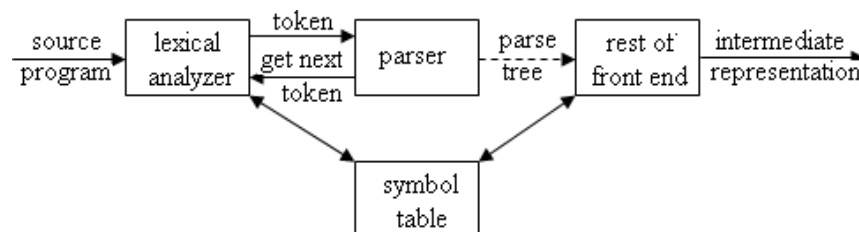
Syntax analysis or parsing is the second phase of a compiler.

A lexical analyzer can identify tokens with the help of regular expressions and pattern rules. But a lexical analyzer cannot check the syntax of a given sentence due to the limitations of the regular expressions.

Regular expressions cannot check balancing tokens, such as parenthesis. Therefore, this phase uses context-free grammar (CFG), which is recognized by push-down automata.

2.1 THE ROLE OF THE SYNTAX ANALYZER

In compiler, the parser obtains a string of tokens from the lexical analyzer and verifies that the string can be generated by the grammar for the source language.



Position of parser in compiler model.

There are three general types of parsers for grammars.

1. **Universal Parsing** methods such as the Cocke-Younger-Kasami algorithm and Earley's algorithm can parse any grammar. But they are too inefficient.
2. **Top-down Parser**, builds parse trees from the top(root) to the bottom(leaves).
3. **Bottom-up Parser**, builds parse trees from the leaves and work up to the root.

2.2. SYNTAX ERROR HANDLING

Programmers frequently write incorrect programs and a good compiler should assist the programmer in identifying and locating errors. Errors can occur in,

- **Lexical**, such as misspelling an identifier, keywords or operator.
- **Syntactic**, such as an arithmetic expression with unbalanced parenthesis.
- **Semantic**, such as an operator applied to an incompatible operand.
- **Logical**, such as an infinitely recursive call.

Often much error detection and recovery is centered around syntax analysis phase. Since many errors are syntactic in nature. Goals of error handler in a parser,

- It should report the presence of errors clearly and accurately.
- It should recover from each error quickly enough to be able to detect subsequent errors.
- It should not significantly slow down the processing of correct programs. Many of the errors could be classified as,

60% punctuation errors
20% operator and operand errors
15% keyword errors
5% other kinds of errors.

1. A common **punctuation error** is to use a comma in place of the semicolon in the argument list of a function declaration. Another is to leave out a semicolon at the end of a line.
2. An example of **operator error** is to leave out the colon from ‘:=’.
3. Misspelling of **keywords** is usually rare like leaving an ‘i’ in “writeln”.

An error handler should report the presence of errors. It should report the place in the source program where an error is detected. A common strategy employed by many compilers is to print the offending line with a pointer to the position at which an error is detected.

2.2.1. Error Recovery Strategies:

There are many different general strategies that a parser can employ to recover from a syntactic error.

Panic Mode Recovery:

This is the simplest method to implement. On discovering an error, the parser discards input symbols one at a time until one of a designated set of synchronizing tokens is found. The synchronizing tokens are usually delimiters such as semicolon or end.

Phrase Level Recovery:

On discovering an error, a parser may perform local correction on the remaining input, i.e., it may replace a prefix of the remaining input by some string that allows the parser to continue. A typical local correction would be to replace a comma by a semicolon, delete an extraneous semicolon or insert a missing semicolon.

Error Productions:

If an error production is used by the parser, we can generate appropriate error diagnostics to indicate the erroneous construct that has been recognized in the input.

Global Correction:

We would like a compiler to make as few changes as possible in processing an incorrect input string. Given an incorrect input string ‘x’ and grammar ‘G’, these algorithms will find a parser tree for a related string ‘y’, such that the number of insertions, deletions and changes of tokens required to transform ‘x’ into ‘y’ is as small as possible.

2.3. REVIEW OF CONTEXT FREE GRAMMARS

A context-free grammar (grammar for short) consists of terminals, nonterminals, a start symbol, and productions.

1. **Terminals** are the basic symbols from which strings are formed. The term "token name" is a synonym for "terminal" and frequently we will use the word "token" for terminal when it is clear that we are talking about just the token name.
2. **Nonterminals** are syntactic variables that denote sets of strings. The nonterminals define sets of strings that help define the language generated by the grammar. They also impose a hierarchical structure on the language that is useful for Both syntax analysis and translation.
3. In a grammar, one nonterminal is distinguished as the **start symbol**, and the set of strings it denotes is the language generated by the grammar. Conventionally, the productions for the start symbol are listed first.
4. The **productions** of a grammar specify the manner in which the terminals and nonterminals can be combined to form strings. Each production consists of:
 - a. A nonterminal called the head or left side of the production; this production defines some of the strings denoted by the head.
 - b. The symbol $\rightarrow$. Sometimes $::=$ has been used in place of the arrow.
 - c. A body or right side consisting of zero or more terminals and nonterminals.

Notational Conventions:

The following notational conventions are,

- These symbols are terminals:
 - Lower-case letters in the alphabet such as a, b, c,...
 - Operator symbols such as +, -, etc.
 - Punctuation symbols such as parentheses, comma etc.
 - The digits 0, 1, ...,9
 - Boldface strings such as **id** or **if**.
- These symbols are nonterminals:
 - Upper-case letters in the alphabet such as A, B, C, ...
 - The letter S, which, when it appears is usually the start symbol.
 - Lower-case italic names such as *expr* and *stmt*.
- Upper-case letters in the alphabet such as X, Y, Z represent grammar symbols that is either nonterminals or terminals.
- Lower-case letters in the alphabet u, v, ...z represent strings of terminals.
- Lower case Greek letters, α , β , γ represents strings of grammar symbols.

Ex: The grammar with the following productions defines simple arithmetic expressions.

$$\begin{aligned} \text{expr} &\rightarrow \text{expr op} \\ \text{expr expr} &\rightarrow \\ (\text{expr}) \end{aligned}$$

$$\text{expr} \rightarrow \text{id}$$

$$\text{op} \rightarrow + \mid - \mid *$$

In this grammar, $V = \{ \text{expr}, \text{op} \}$, $T = \{ \text{id}, (,), +, -, * \}$, and $S = \text{expr}$.

2.3.1. DERIVATION AND PARSE TREES

The construction of a parse tree can be made precise by taking a derivational view, in which productions are treated as rewriting rules.

Beginning with the start symbol, each rewriting step replaces a nonterminal by the body of one of its productions.

For example, consider the following grammar, with a single nonterminal E :

$$E \rightarrow E + E \mid E * E \mid - E \mid (E) \mid \text{id}$$

The production $E \rightarrow - E$ signifies that if E denotes an expression, then $- E$ must also denote an expression. The replacement of a single E by $- E$ will be described by writing $E \Rightarrow -E$ which is read, "E derives $- E$."

The production $E \rightarrow (E)$ can be applied to replace any instance of E in any string of grammar symbols by (E) , e.g., $E * E \Rightarrow (E) * E$ or $E * E \Rightarrow E * (E)$

We can take a single E and repeatedly apply productions in any order to get a sequence of replacements. For example, $E \Rightarrow - E \Rightarrow - (E) \Rightarrow - (\text{id})$

We call such a sequence of replacements a derivation of $-(\text{id})$ from E . This derivation provides a proof that the string $-(\text{id})$ is one particular instance of an expression.

Leftmost And Rightmost Derivation Of A String

 **Leftmost derivation** – A leftmost derivation is obtained by applying production to the leftmost variable in each step.

 **Rightmost derivation** – A rightmost derivation is obtained by applying production to the rightmost variable in each step.



Example 1:

Grammar with the following productions defines simple expressions. Verify the string $1+2;$ is a sentence of grammar,

$$\text{stmt} \rightarrow \text{expr semi}$$

$$\text{expr} \rightarrow \text{factor op expr} \mid$$

$$\text{factor factor} \rightarrow \text{number}$$

$$\text{op} \rightarrow + \mid - \mid * \mid /$$

$$\text{semi} \rightarrow ;$$

$$\text{number} \rightarrow 0 \mid 1 \mid 2 \mid 3$$

Soln:

$$\text{Stmt} \rightarrow \text{expr semi}$$

$$\rightarrow \text{factor op expr semi}$$

$\rightarrow$ number op expr semi
 $\rightarrow$ 1 op expr semi
 $\rightarrow$ 1 + expr semi
 $\rightarrow$ 1 + factor semi
 $\rightarrow$ 1 + number semi
 $\rightarrow$ 1 + 2 semi
 $\rightarrow$ 1 + 2 ;

Example 2 :

Let any set of production rules in a CFG be

$$X \rightarrow X+X \mid X*X \mid X \mid a$$

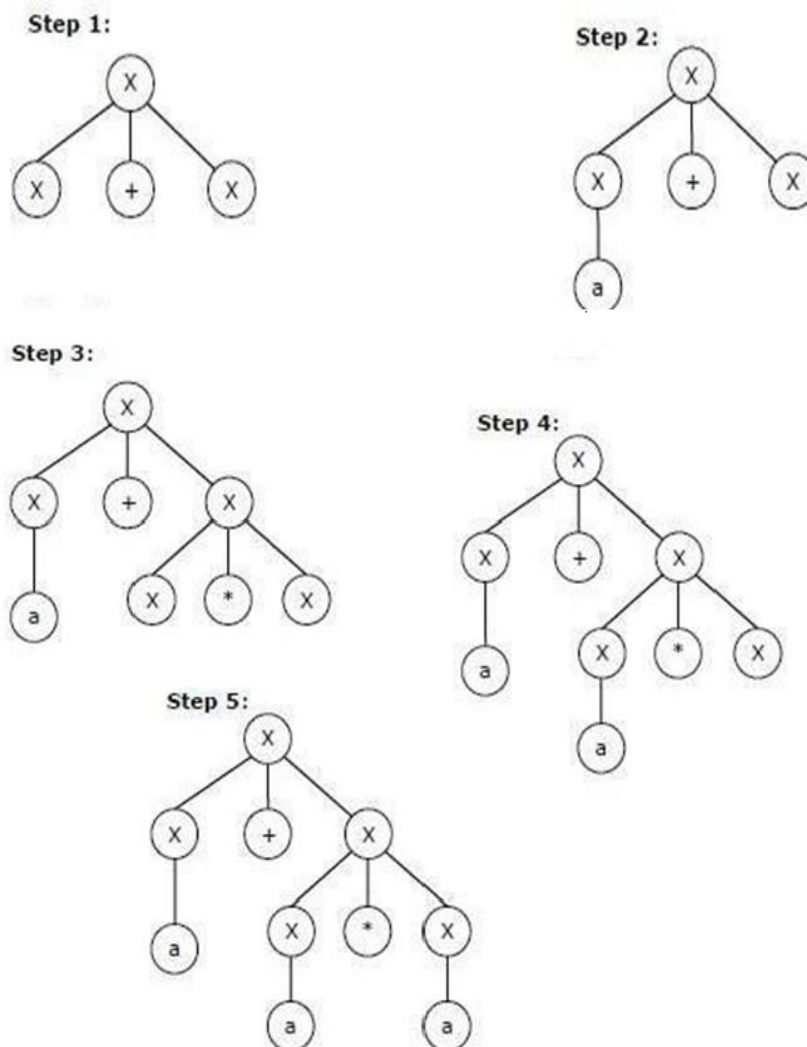
over an alphabet $\{a\}$.

The leftmost derivation for the string "a+a*a" may be –

$$X \rightarrow X+X \rightarrow a+X \rightarrow a + X*X \rightarrow a+a*X \rightarrow a+a*a$$

Soln:

The step wise derivation of the above string is shown as below –



The rightmost derivation for the above string "a+a*a" may be

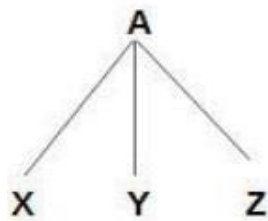
$$X \rightarrow X * X \rightarrow X * a \rightarrow X + X * a \rightarrow X + a * a \rightarrow a + a * a$$

PARSE TREES

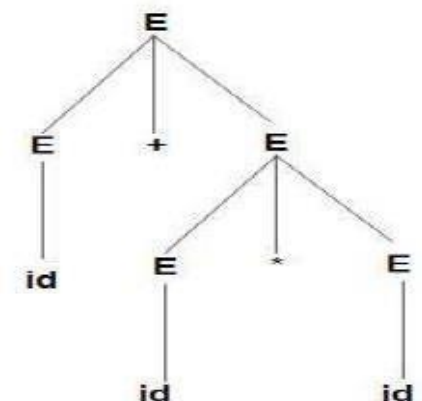
- Parse tree is a hierarchical structure which represents the derivation of the grammar to yield input strings.
- Simply it is the graphical representation of derivations.
- Root node of parse tree has the start symbol of the given grammar from where the derivation proceeds.
- Leaves of parse tree are labeled by non-terminals or terminals.

Each interior node is labeled by some non terminals.

- If $A \rightarrow xyz$ is a production, then the parse tree will have A as interior node whose children are x , y and z from its left to right.



- Construct parse tree for $E \rightarrow E + E / E * E / id$



Yield Of Parse Tree

- The leaves of the parse tree are labeled by nonterminal or terminals and read from left to right, they constitute a sentential form, called the yield or frontier of the tree.
- Figure above represents the parse tree for the string **id+ id*id**. The string **id + id * id**, is the yield of parse tree depicted in Figure.

AMBIGUITY

- An ambiguous grammar is one that produces more than one leftmost or more than one rightmost derivation for the same sentence.
- For most parsers, it is desirable that the grammar be made unambiguous, for if it is not, we cannot uniquely determine which parse tree to select for a sentence.
- In other cases, it is convenient to use carefully chosen ambiguous grammars, together with disambiguating rules that "throw away" undesirable parse trees, leaving only one tree for each sentence.

EXAMPLE

Show that the following grammar is ambiguous, $E \rightarrow E + E \mid E * E \mid (E) \mid - E \mid id$

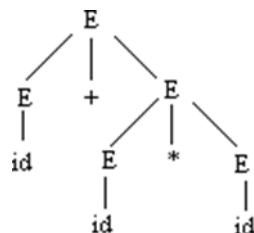
1st Leftmost Derivation

$E \Rightarrow E + E$
 $\Rightarrow id + E$
 $\Rightarrow id + E * E$
 $\Rightarrow id + id * E$
 $\Rightarrow id + id * id$

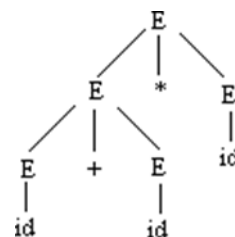
2nd Leftmost Derivation

$E \Rightarrow E * E$
 $\Rightarrow E + E * E$
 $\Rightarrow id + id * E$
 $\Rightarrow id + id * E$
 $\Rightarrow id + id * id$

1st Parse Tree



2nd Parse Tree



2.4. BASIC PARSING APPROCHES

Grammars are capable of describing most, but not all, of the syntax of programming languages.

Every grammar should have the following;

1. Eliminating Ambiguity
2. Elimination of Left Recursion
3. Left Factoring

1. Eliminating Ambiguity:

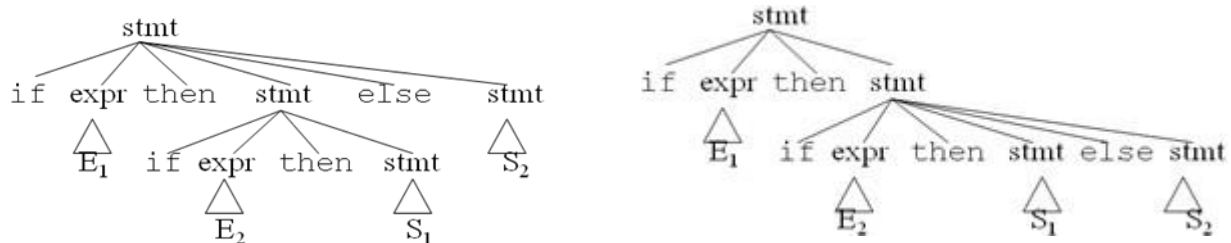
- For the most parsers, the grammar must be unambiguous.
- We should eliminate the ambiguity in the grammar during the design phase of the compiler.
- An unambiguous grammar should be written to eliminate the ambiguity.

Example:

We shall eliminate the ambiguity from the following “dangling-else” grammar,

$$\begin{aligned} \text{stmt} \rightarrow & \text{if expr then stmt} \\ & | \text{if expr then stmt else stmt} \\ & | \text{other} \end{aligned}$$
Soln:

The above grammar is ambiguous. Since the string “if E₁ then if E₂ then S₁ else S₂” can lead to two parse trees which are show below;



The disambiguating rule can be incorporated directly into the grammar. So we can rewrite the grammar. The unambiguous grammar will be:

$$\begin{aligned} \text{stmt} &\rightarrow \text{matchedstmt} \mid \text{unmatchedstmt} \\ \text{matchedstmt} &\rightarrow \text{if expr then matchedstmt else matchedstmt} \mid \text{otherstmt} \\ \text{unmatchedstmt} &\rightarrow \text{if expr then stmt} \mid \text{if expr then matchedstmt else unmatchedstmt} \end{aligned}$$
2. Elimination of Left Recursion:

(i) A grammar is **left recursive** if it has a non-terminal A such that there is a derivation.

$A \Rightarrow A\alpha$ for some string α .

- Top-down parsing techniques **cannot** handle left-recursive grammars.
- So, we have to convert our left-recursive grammar into an equivalent grammar which is not left- recursive.
- The left-recursion may appear in a single step of the derivation (*immediate left-recursion*), or may appear in more than one step of the derivation.

$$A \rightarrow A\alpha \mid \beta$$

$$\Downarrow$$

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \epsilon$$
General Rule for Eliminating immediate left recursion:

- Group the productions as $A \rightarrow A\alpha_1 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \dots \mid \beta_n$.
- Replace the A – productions by,

$$A \rightarrow \beta_1 A' \mid \dots \mid \beta_n A'$$

$$A' \rightarrow \alpha_1 A' \mid \dots \mid \alpha_m A' \mid \epsilon$$
Example 1:

Consider the following grammar, to eliminate left recursion,

$$E \rightarrow E+T \mid T$$

$$T \rightarrow T*F \mid F$$

$$F \rightarrow (E) \mid \text{id}$$

Soln:

Eliminating left recursion,

we get, $E \rightarrow T E'$

$E' \rightarrow +T E' \mid \varepsilon$

$T \rightarrow F T'$

$T' \rightarrow *F T' \mid \varepsilon$

$F \rightarrow \text{id} \mid (E)$

Example 2:

Eliminate left recursion from the grammar,

$S \rightarrow (L) \mid a$

$L \rightarrow L, S \mid S$

Soln:

Eliminate left recursion, we get,

$S \rightarrow (L) \mid a$

$L \rightarrow SL'$

$L' \rightarrow ,SL' \mid \varepsilon$

3. Left Factoring:

Left factoring is a grammar transformation that is useful for producing a grammar suitable for predictive parsing. The basic idea is that when it is not clear which of two alternative productions to use to expand a non-terminal A, we may be able to rewrite the A-productions to defer the decision until we have seen enough of the input to make the right choice.

General rule for Left Factoring a Grammar:

- Given two A – productions are,

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$$
- The input begins with a nonempty string derived from α , when processing α we cannot know whether expand A to $\alpha\beta_1$ or A to $\alpha\beta_2$
- But, if we re-write the grammar as follows

$$A \rightarrow \alpha A'$$

$$A' \rightarrow \beta_1 \mid \beta_2$$

Example:

Left factor the following

grammar, $S \rightarrow \text{iEtS} \mid$

$\text{iEtSeS} \mid a$

$E \rightarrow b$

Soln:

Left factored grammar is,

$S \rightarrow \text{iEtSS}' \mid a$

$S' \rightarrow \text{eS} \mid \varepsilon$

$E \rightarrow b$

2.5. TOP DOWN PARSING

✚ Parsing is the process of determining if a string of token can be generated by a grammar.

✚ Mainly 2 parsing approaches:

➤ **Top Down Parsing**

➤ **Bottom Up Parsing**

✚ In **top down parsing**, parse tree is constructed from top (root) to the bottom (leaves).

✚ In **bottom up parsing**, parse tree is constructed from bottom (leaves)) to the top (root).

✚ It can be viewed as an attempt to construct a parse tree for the input starting from the root and creating the nodes of parse tree in preorder.

✚ Pre-order traversal means: 1. Visit the root 2. Traverse left subtree 3. Traverse right subtree.

Top-down parsing find a leftmost derivation for an input string. It also construct a parse tree for the input starting from the root and creating the nodes of the parse tree in preorder. There are two types of top- down parsing. They are,

1. Recursive Descent Parsing
2. Predictive Parsing

2.5.1. RECURSIVE DESCENT PARSING

Recursive Descent Parsing is a general form of top-down parsing, called recursive descent that may involve backtracking, i.e., making repeated scans of the input.

Example: Backtracking

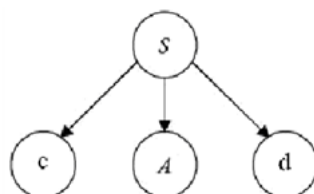
Consider the grammar,

$S \rightarrow cAd$

$A \rightarrow ab \mid a$ and the input string $w = cad$. To construct a parse tree for the input string 'cad' top-down.

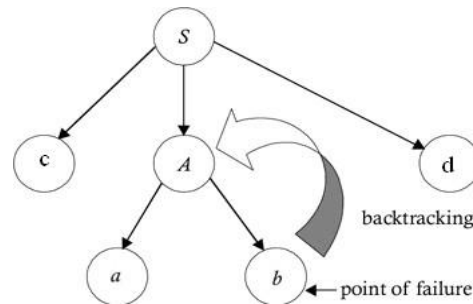
Soln:

- (i) Create a tree consisting of a single node labeled S. An input pointer to 'c', the first symbol of 'w'.
- (ii) Use the first production for 'S' to expand the tree and obtain the tree.



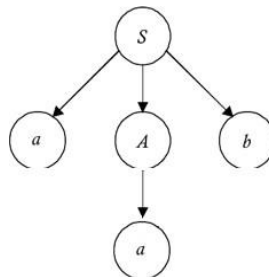
The leftmost leaf labeled 'c', matches the first symbol of 'w', advance the input pointer to the second symbol of 'w', i.e., 'a'.

- (iii) Expand the next leaf of tree i.e., 'A' using the first alternative for 'A' to obtain the tree.



The next to leftmost leaf labeled 'a', matches the second of 'w', advance the input pointer to the third symbol of 'w', i.e., 'd'.

- (iv) Compare the input symbol 'd' against the next leaf labeled 'b'. Since 'b' does not match 'd', we report failure and go back to 'A'(backtracking) to see whether there is another alternative for 'A' that we have not tried. In going back to 'A', we must reset the pointer to position 2. i.e., 'a'.
- (v) Now expand the 'A' using the second alternative for 'A' to obtain the tree.

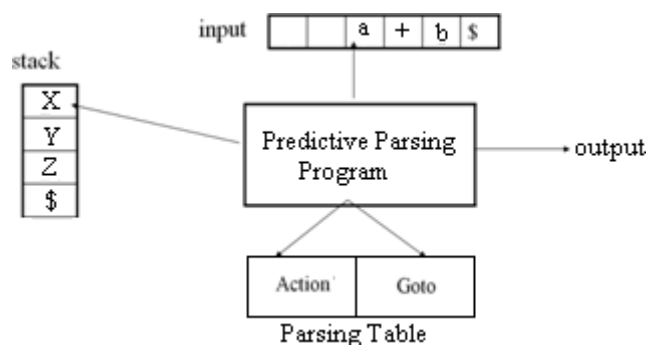


2.5.2. PREDICTIVE PARSING

Predictive Parsing is a special form of Recursive Descent parsing without backtracking. Predictive Parser is also known as LL(1) parser. To construct a predictive parser, we must know given the current input symbol 'a' and the nonterminal 'A' to be expanded, which one of the alternatives of production $A \rightarrow \alpha_1 \mid \dots \mid \alpha_n$ is the unique alternative that derives a string beginning with 'a'.

Nonrecursive Predictive Parsing:

Non-Recursive predictive parsing is a table-driven parser. The key problem during predictive parsing is that of determining the production to be applied for a nonterminal. To overcome the problem found in predictive parsing, build a nonrecursive predictive parser by maintaining a stack explicitly rather than implicitly via recursive calls.



Model of a nonrecursive predictive parser.

The nonrecursive parser looks up the production to be applied in a parsing table. A table driven predictive parser has an input buffer, a stack, a parsing table and an output stream.

- The **input buffer** contains the string to be parsed followed by \$, a symbol used as a right end marker to indicate the end of the input string.
- The **stack** contains a sequence of grammar symbols with \$ on the bottom, indicating the bottom of the stack. Initially, the stack contains the start symbol of the grammar on top of \$.
- The **parsing table** is a two-dimensional array $M[A, a]$, where 'A' is a nonterminal and 'a' is a terminal or the symbol \$.
- The **output stream**, a production rule representing a step of the derivation sequence (left-most derivation) of the string in the input buffer.

Behaviour of Parser:

The program considers symbol 'X' at the top of the stack and the current symbol in the input string 'a'.

These two symbols determine the parser action. There are four possible parser actions.

1. If X and a are \$ → parser halts (successful completion)
2. If X and a are the same terminal symbol (different from \$)
→ parser pops X from the stack, and moves the next symbol in the input buffer.
1. If X is a non-terminal
→ parser looks at the parsing table entry $M[X, a]$. If $M[X, a]$ holds a production rule, $X \rightarrow UVW$, it pops X from the stack and pushes WVU into the stack.
4. none of the above → error
 - all empty entries in the parsing table are errors.
 - If X is a terminal symbol different from a, this is also an error case.

Algorithm for Nonrecursive Predictive Parsing:

Input : A string 'w' and a parsing table 'M' for grammar 'G'.

Output : If 'w' is in $L(G)$, accept, otherwise an error indication.

Method : Initially the parser is in a configuration in which it has '\$S' on the stack with 'S', the start symbol of 'G' on top and 'w\$' in the input buffer.

Program:

```

set ip to point the first symbol
of w$; repeat
  Let X be the top stack symbol and 'a' be the symbol pointed
  to by ip; if X is a terminal or $ then
    if X = a then
      Pop X from the stack and
      advance ip; else error();
    else /* X is a nonterminal */
      if  $M[X, a] = X \rightarrow Y_1 Y_2 \dots Y_k$  then
        begin Pop X from the
        stack;
        Push  $Y_k Y_{k-1}, \dots, Y_1$  onto the stack, with  $Y_1$ 
        on top; Output the production  $X \rightarrow Y_1 Y_2 \dots Y_k$ 
      end
    else error();
until X = $ /* stack is empty */

```

Construction of Predictive Parsing Tables:

Two functions are used in the construction of LL(1) parsing tables:

- FIRST and FOLLOW

FIRST and FOLLOW:

The construction of a predictive parser is aided by two functions associated with a grammar G. These functions are used to fill entries of predictive parsing table for G.

FIRST:

- If X is a terminal symbol then $\text{FIRST}(X) = \{X\}$
- If $X \rightarrow \epsilon$ is a production, then add ϵ to $\text{FIRST}(X)$.
- If X is a non-terminal symbol and $X \rightarrow Y_1 Y_2 \dots Y_n$ is a production, then $\text{FIRST}(X)$ is in $\text{FIRST}(Y_1)$ and if 'a' is in $\text{FIRST}(Y_1)$, then $\text{FIRST}(X)$, $\text{FIRST}(Y_1)$ is in $\{a\}$.

FOLLOW:

To compute $\text{FOLLOW}(A)$ for all nonterminals 'A', apply the following rules.

1. If S is the start symbol, then \$ is in $\text{FOLLOW}(S)$
2. if $A \rightarrow \alpha B \beta$ is a production rule, then everything in $\text{FIRST}(\beta)$ [where 'β' is nonterminal] is $\text{FOLLOW}(B)$ except ϵ . If ϵ is in $\text{FIRST}(\beta)$ then everything in $\text{FOLLOW}(\beta)$ is in $\text{FOLLOW}(B)$.

If $A \rightarrow \alpha B$ is a production rule, then everything in $\text{FOLLOW}(A)$ is in $\text{FOLLOW}(B)$.

Algorithm: Construction of Predictive Parsing Table:

Input : Grammar G.

Output : Parsing table M.

Method:

1. For each production rule $A \rightarrow \alpha$ of a grammar G
2. For each terminal a in $\text{FIRST}(\alpha)$, add $A \rightarrow \alpha$ to $M[A, a]$
3. If ϵ is in $\text{FIRST}(\alpha)$, add $A \rightarrow \alpha$ to $M[A, a]$ for each terminal a in $\text{FOLLOW}(A)$
4. If ϵ in $\text{FIRST}(\alpha)$ and \$ is in $\text{FOLLOW}(A)$, add $A \rightarrow \alpha$ to $M[A, \$]$.
5. Make each undefined entry of M be error.

LL(1) Grammars:

A grammar G whose parsing table has no multiple defined entries can be called LL(1) grammar. The first L indicates scanning the input from left to right and second L to indicate producing left most derivation. (1) indicate one input symbol of lookahead at each step to make parsing action decision.

Ex.1:

Construct a predictive parser for the grammar,

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \text{id} \text{ and parse the string 'id + id * id'.$$

Soln:

Step 1: To eliminate left recursion or left factoring. Here eliminate left recursion, after

eliminating left recursion from the above grammar, we get,

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \varepsilon$$

$$F \rightarrow (E) \mid \text{id}$$

Step 2: To compute FIRST and FOLLOW.

Find FIRST:

$$\text{FIRST}(E) = \text{FIRST}(T) = \text{FIRST}(F) = \{ (, \text{id} \}$$

$$\text{FIRST}(E') = \{ +, \varepsilon \}$$

$$\text{FIRST}(T') = \{ *, \varepsilon \}$$

Find FOLLOW:

$$\text{FOLLOW}(E) = \{ \$,) \}$$

$$\text{FOLLOW}(E') = \text{FOLLOW}(E) = \{ \$,) \}$$

$$\text{FOLLOW}(T) = \{ +,), \$ \}$$

$$\text{FOLLOW}(T') = \text{FOLLOW}(T) = \{ +,), \$ \}$$

$$\text{FOLLOW}(F) = \{ +, *,), \$ \}$$

Step 3: To construct Parsing table.

	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow \text{id}$			$F \rightarrow (E)$		

Step 4: Implementation of stack.

<u>stack</u>	<u>input</u>	<u>output</u>
\$E	id+id*id\$	$E \rightarrow TE'$
\$E'T	id+id*id\$	$T \rightarrow FT'$
\$E' T'F	id+id*id\$	$F \rightarrow \text{id}$
\$ E' T'id	id+id*id\$	pop 'id'
\$ E' T'	+id*id\$	$T' \rightarrow \varepsilon$
\$ E'	+id*id\$	$E' \rightarrow +TE'$
\$ E' T+	+id*id\$	pop '+'
\$ E' T	id*id\$	$T \rightarrow FT'$
\$ E' T' F	id*id\$	$F \rightarrow \text{id}$
\$ E' T'id	id*id\$	pop 'id'
\$ E' T'	*id\$	$T' \rightarrow *FT'$
\$ E' T'F*	*id\$	pop '*'
\$ E' T'F	id\$	$F \rightarrow \text{id}$
\$ E' T'id	id\$	pop 'id'
\$ E' T'	\$	$T' \rightarrow \varepsilon$
\$ E'	\$	$E' \rightarrow \varepsilon$
\$	\$	accept

Ex.2:

Check whether the following grammar is a LL(1) grammar.

$$S \rightarrow iEtS \mid iEtSeS \mid a$$

$$E \rightarrow b$$

Soln:

Step 1: To eliminate left recursion or left factoring. Here left factoring the grammar, after left factoring from the above grammar, we get,

$$S \rightarrow iEtSS' \mid a$$

$$S' \rightarrow eS \mid \epsilon$$

$$E \rightarrow b$$

Step 2: To compute FIRST and FOLLOW.

- Find FIRST:

$$\text{FIRST}(S) = \{i, a\}$$

$$\text{FIRST}(S') = \{e, \epsilon\}$$

$$\text{FIRST}(E) = \{b\}$$

Find FOLLOW

$$\text{FOLLOW}(S) = \{ \$, e \}$$

$$\text{FOLLOW}(S') = \text{FOLLOW}(S) = \{ \$, e \}$$

$$\text{FOLLOW}(E) = \{ t \}$$

Step 3: To construct Parsing table.

	a	b	i	t	E	\$
S	$S \rightarrow a$		$S \rightarrow iEtSS'$			
S'					$S' \rightarrow eS', S' \rightarrow \epsilon$	$S' \rightarrow \epsilon$
E		$E \rightarrow b$				

Table contains multiple defined entry in $M[S', e]$, therefore this grammar is not LL(1) grammar.

Error Recovery in Predictive Parsing:

An error may occur in the predictive parsing (LL(1) parsing)

- if the terminal symbol on the top of stack does not match with the current input symbol.
- if the top of stack is a non-terminal A, the current input symbol is a, and the parsing table entry $M[A, a]$ is empty.

** Panic-Mode Error Recovery**

- Skipping the input symbols until a synchronizing token is found.
- All the empty entries are marked as **sync** to indicate that the parser will skip all the input symbols until a symbol in the follow set of the non-terminal A which on the top of the stack. Then the parser will pop that non-terminal A from the stack. The parsing continues from that state.
- To handle unmatched terminal symbols, the parser pops that unmatched terminal symbol from the stack and it issues an error message saying that that unmatched terminal is inserted.

Ex:

$$S \rightarrow AbS \mid e \mid \epsilon$$

$$A \rightarrow a \mid cAd$$

$$\text{FOLLOW}(S) = \{ \$ \}$$

$$\text{FOLLOW}(A) = \{ b, d \}$$

	a	b	c	d	e	\$
S	$S \rightarrow AbS$	sync	$S \rightarrow AbS$	sync	$S \rightarrow e$	$S \rightarrow \epsilon$
A	$A \rightarrow a$	sync	$A \rightarrow cAd$	sync	sync	sync

<u>stac</u>	<u>input</u>	<u>output</u>	<u>stack</u>	<u>input</u>	<u>output</u>
\$S	aab\$	$S \rightarrow AbS$	\$S	ceadb\$	$S \rightarrow AbS$
\$SbA	aab\$	$A \rightarrow a$	\$SbA	ceadb\$	$A \rightarrow cAd$
\$Sba	aab\$		\$SbdAc	ceadb\$	
\$Sb	ab\$	Error: missing b, inserted	\$SbdA	eadb\$	Error:unexpected e (illegal A)
			(Remove all input tokens until first b or d, pop A)		
\$S	ab\$	$S \rightarrow AbS$	\$Sbd	db\$	
\$SbA	ab\$	$A \rightarrow a$	\$Sb	b\$	
\$Sba	ab\$		\$S	\$	$S \rightarrow \epsilon$
\$Sb	b\$		\$	\$	accept
\$S	\$	$S \rightarrow \epsilon$			
\$	\$	Accept			

Phrase-Level Error Recovery

- (i) Each empty entry in the parsing table is filled with a pointer to a specific error routine to take care that error case.
 - (ii) These error routines may:
 - (iii) change, insert, or delete input symbols.
 - (iv) issue appropriate error messages
 - (v) pop items from the stack.
 - (vi) We should be careful when we design these error routines, because we may put the parser into an infinite loop.
-